axiateam
RDT company

# Módulo 10

## Módulos

Curso: TypeScript

60 min

# Objetivos de Aprendizaje

- Comprender el **sistema de módulos** de TypeScript
- Organizar código en **módulos lógicos**
- Utilizar **rutas absolutas y relativas**
- Configurar **paths** en tsconfig.json

# 1. Contexto

Organización del Código

# Por qué Módulos

Los módulos organizan el código en unidades reutilizables:

```
src/
├── types/
│   └── index.ts      # Tipos compartidos
├── services/
│   └── taskService.ts # Lógica de negocio
├── utils/
│   └── helpers.ts    # Utilidades
└── index.ts          # Punto de entrada
```

# 2. Estructura de Módulos

# Módulos en TypeScript

Los módulos organizan el código en archivos independientes con export e import.

```
// src/types/index.ts
export type TaskStatus = "pending" | "inProgress" | "completed";

export interface Task {
  id: string;
  title: string;
  status: TaskStatus;
}

export interface User {
  id: string;
  name: string;
}
```

```
// src/services/taskService.ts
import { Task, TaskStatus } from "../types";

export class TaskService {
  private tasks: Task[] = [];

  addTask(task: Task): void {
    this.tasks.push(task);
  }
```

```
  getTasks(): Task[] {
    return [...this.tasks];
  }
}
```

# Lab: Organizar Sistema de Tareas

**Objetivo:** Estructurar el proyecto en módulos.

1. Crea estructura de carpetas
2. Exporta tipos en `src/types/index.ts`
3. Crea modelo en `src/models/task.ts`
4. Implementa servicio en `src/services/taskService.ts`

# Código del Lab

```
// Estructura de carpetas
// src/
// ├── types/index.ts
// ├── models/task.ts
// ├── services/taskService.ts
// └── index.ts

// src/types/index.ts
export type TaskStatus = "pending" | "inProgress" | "completed";
export type TaskPriority = "low" | "medium" | "high";

export interface Task {
  id: string;
  title: string;
  description?: string;
  status: TaskStatus;
  priority: TaskPriority;
  assigneeId?: string;
  dueDate?: Date;
```

# 3. Export e Import Avanzado

# export default vs export named y Re-exports

export default para una sola exportación principal; export named para múltiples. Re-exports simplifican barrel files.

```
// export named: varios exports
export type TaskStatus = "pending" | "inProgress";
export interface Task { id: string; title: string; }

// export default: un solo export principal
export default class TaskService { }

// Re-export desde barrel
export { Task, TaskStatus } from "./types";
```

# import type

import type solo importa tipos, eliminando el código en runtime.

```
// Solo tipos: se elimina en compilación
import type { Task, User } from "./types";

// Import mixto (valor + tipo)
import { Task } from "./types";
import type { TaskStatus } from "./types";
```

# 3. Rutas y Paths

# Path Mapping

Configura rutas limpias en `tsconfig.json`:

```
{
  "compilerOptions": {
    "baseUrl": "./src",
    "paths": {
      "@types/*": ["types/*"],
      "@models/*": ["models/*"],
      "@services/*": ["services/*"],
      "@utils/*": ["utils/*"]
    }
  }
}
```

Uso:

```
import { Task } from "@types";
import { TaskModel } from "@models/task";
import { TaskService } from "@services/taskService";
```

# Lab: Configurar Paths

**Objetivo:** Configurar paths para el proyecto.

1. Actualiza `tsconfig.json`
2. Configura `baseUrl`
3. Define los paths
4. Verifica imports con alias

# Código del Lab

```
{
  "compilerOptions": {
    "baseUrl": "./src",
    "paths": {
      "@/*": ["./*"]
    }
  }
}
```

```
// Con path mapping
import { Task } from "@/types";
import { TaskModel } from "@/models/task";
import { TaskService } from "@/services/taskService";

// Sin path mapping (relativo)
import { Task } from "../types";
import { TaskModel } from "../models/task";
import { TaskService } from "../services/taskService";
```

# Resumen

# Puntos Clave

- Los **módulos** organizan el código en unidades reutilizables
- **export** hace disponible código fuera del módulo
- **import** consume código de otros módulos
- **Paths** permiten imports más limpios
- **baseUrl** define la ruta raíz para los paths

# Ejercicio Integrador

Sistema de Biblioteca

# Caso 2: Biblioteca

**Objetivo:** Organizar el sistema de biblioteca en módulos.

1. Crea estructura de carpetas por dominio
2. Organiza tipos, modelos y servicios
3. Configura paths personalizados

**Trabajo en grupos de 2-4 personas.**

# Tips y Gotchas

## ✅ Tips

- Path mapping (`paths` en tsconfig) evita imports relativos largos
- Barrel files (index.ts) simplifican imports
- Un archivo = una responsabilidad

## ⚠️ Gotchas

- Path mapping solo funciona en compilación, no en runtime
- Evita imports circulares
- `baseUrl` debe apuntar al root del proyecto

# Referencias

## Recursos Adicionales

- Modules
- Module Resolution

# Gracias!

## Módulo 10 Completado

Siguiente: Módulo 11 - Importaciones y Exportaciones